Fox's Algorithm using MPI 2026/27

Fábio Semedo Gabriel Carlos Gustavo dos Santos

Contents

1 Introduction **1**

2 Implementation **1**
2.1 Distance Product 2
2.2 Fox's Algorithm 2
2.3 Optimizations 3

3 Experiments **4**
3.1 Personal Computer 4
3.2 Distributed Machines 5

4 Conclusion **6**

1 Introduction

For this project, we were meant to explore a distributed memory implementation of Fox's algorithm. We have made some experiments using a personal computer of ours and two different configurations—per-node and per-slot—using the distributed machines at *DCC*. In Chapter 2, we describe our implementations of the Distance Product algorithm and of Fox's Algorithm along with the path that led to their completion. Then, in Chapter 3 we make an experimental analysis, using measurements for a sequential implementation and the first 5 positive, non-zero, square numbers of processes. Lastly, in Chapter 4 we make some final remarks about what we accomplished and what we could have accomplished.

2 Implementation

The implementation is split among four modules, of which a main file and two external header files, respectively: "*src/main.c*", "*src/Math/Math.h*" and "*src/Misc/Misc.h*". The descriptions of each part of the implementation will cover the motivation as well as the reasoning behind each of these files. Generically, it can be said that the core logic of the algorithms can be found within "*main.c*", whereas the external header files contain helper functions. Some of these helper functions are optimizations of other functions[1].

The implementation has been extensively optimized. Many of these optimizations consist of giving more precise information to the compiler, while others take advantage of the conventional behaviour of the *CPU* and of the impact the representation of data structures has on efficiency. As most optimizations are out of the scope of this project, only the most notable ones will be considered. Naturally, any optimization that directly regards distributed memory will also be considered.

[1]Some of these functions are from the standard library of C, although the implementation may differ from OS to OS. Others are functions we had previously implemented, but were eventually optimized, such as *matrixMultiply*.

2.1 Distance Product

The implementation for the matrix multiplication algorithm was mostly direct. First, essential data structures were made, along with a rudimentary parser, so that input matrices could be represented within the program after being received as input. Then, the pseucocode that was provided[2] for the *matrixMultiply* function was converted into C and given all due modifications that were seen as appropriate. For instance, the number of steps for the squaring method is calculated seperately as its value can be reused. Another change that was made to the implementation of the algorithm had to do with *cache locality*. In particular, one can make better use of the cache, since, everytime an element of the matrix is accessed, the *CPU* loads a chunk of what follows the array that the element inhabits within the matrix. By having both the original and the transposed matrices, the entire innermost loop of *matrixMultiply* benefits from not having to constantly fetch the element from outside the cache, as a consequence of strided accesses.

This step of the implementation did pose one minor difficulty: it had not become clear to us that 0's that rest outside of the main diagonal of the matrix had to be represented as an infinity. Indeed, this is an obvious requirement, as such paths are meant to represent currently impossible direct paths, and the matrix multiplication algorithm that we used relied on determining a minimal[3] weight. Better yet, the description of the algorithm that was provided to us did specify that, if the weight at (i, j) is 0, for $i \neq j$, then said weight must be converted to infinity.

Lastly, in order to reduce the time complexity of our algorithm for matrix multiplication, we used the squaring method. Given that we want to determine $matrixMultiply^{n-1}$ and that its result will not change if we increase it further, we can simply compute it using fast exponentiation[4].

2.2 Fox's Algorithm

The implementation of the Fox Algorithm was also mostly straight forward. It revolves around dividing a matrix into multiple, smaller, submatrices. The number of processes must be a square number such that its square root is able to divide the matrix into as many submatrices as there are processes, so that each process has its own square submatrix. The resulting distribution of submatrices can be thought of as an entire matrix, where the elements are submatrices. Each process stores two submatrixes, A and B, which are equal initially. Then, the following algorithm is run as many times as there are submatrices. From each row, a process is chosen[5] and it broadcasts its submatrix A to the other processes of the same row[6] including itself, so that they can be multiplied each process' submatrix B using the *matrixMultiply* function. The result from the submatrix multiplication within each process is stored as the new submatrix A of each process, and the submatrix B of each process is then sent to the process that lies in the row above, in

[2] https://users.cecs.anu.edu.au/~Alistair.Rendell/Teaching/apac_comp3600/module4/all_pairs_shortest_paths.xhtml

[3] We call it minimal, although it eventually becomes the minimum weight thanks to the guarantee that multiplying the matrix by itself $n-2$ times—where n is the number of nodes or elements of the matrix—results in the matrix representing the shortest paths. Also, it should be mentioned that a path connecting n nodes requires $n-1$ edges, and that multiplying a natural number n' by itself twice is the same as cubing n'. Hopefully it is clear why $n-2$ is the magic number.

[4] This makes use of the number theoretic concept of binary representation of powers. Concretely, it means that we must first compute M, then M^2, then M^4 and so on. If M^7 is needed, for example, it can be computed by first computing M, then M^2 as M times M, and, finally, M^4 as M^2 times M^2. In the end, one must multiply M^4, M^2 and M, thus reducing the number of multiplications from 7 to 4. This is more powerful the larger the number is, as it goes from $O(n)$ to $O(log(n))$. In our case, we do not even need to do the final calculation, as M^{128} is equal to M^{100} if M^{100} is the stabilizing goal.

[5] There is a formula for making this choice within each row. The column that is selected is indexed by $(r+i) mod sqrt(p)$, where r is the index of the row, i is the number of the current iteration—that goes from 0 to $sqrt(p)$—and $sqrt(p)$ is the square root of p, the number of processes.

[6] Keep in mind that there is a 1-to-1 correspondence between processes and submatrices.

the same column, which eventually stores the received submatrix as the new submatrix B. Before our implementation of the algorithm begins, there is a sharing of submatrices in row order and a broadcaster order is defined. Given that our *matrixMultiply* function expects a transposed matrix for its second parameter, everytime a new submatrix is received from the row communication, the current process transposes it. In the end, all submatrices are sent back to the root process. The final matrix is composed by the submatrices A, from each process, in the same order as the initial submatrices were distributed to the processes.

2.3 Optimizations

Optimization happened in a few steps. First, we use transposed matrixes for better cache locality. In spite of it being $O(n^2)$, the improved locality outweighs the calculation costs for large matrices. With this, we also refactored all the code to expect the matrices as 1D arrays. It is natural that if a 2D array of size n is represented as $x[0], x[1], ..., x[n]$, where each $x[i]$ is a pointer to an array of size n. Thus, despite belonging to the same variable in the code, each one of these n arrays will be placed in a different place in memory, usually in a non-contiguous way, resulting in worse cache locality and, thus, worsening performance.

Afterwards we turned to compiler auto-vectorization for speedup, using the compilation *"flag-fopt-info-vec-missed"*, to find the places where our code was not vectorization-friendly. Things like for loops with indexes of type int instead of size$_t affectvectorizationduetothepossibilityo$

A compiler could apply the optimizations mentioned in the previous statement automatically, although -O3 is infamous for creating undefined-behaviour when optimizing. As such, we looked for other ways to give hints to the compiler and we found out about the *restrict* keyword. The purpose of this keyword is to tell the compiler that no two pointers point to the same address and use it at the same time—also known as pointer aliasing.

Although this appears to alias the matrix pointer, not accessing the base matrix variable within *mRow*'s scope makes it valid and aids in compiler analysis. Together, these general C optimizations yielded a tenfold speedup.

To further improve performance, we implemented vectorization manually. Firstly, we change our matrix allocation function to align all pointers to a 32-bit memory address, *SIMD* instructions have a aligned and non-aligned version for loads and stores so if we can, using aligned versions are always faster.

With this new allocation function, we create *SIMD* helper functions where we load our data to *SIMD* (Single Instruction Multiple Data) registers and do *memcopy* or *memmove* and others. As for our *matrixMultiply*, we load the pointer to a *SIMD* register and we perform all calculations with these registers (addition, multiplication, finding the minimum between 2 *SIMD* registers, etc) then we store it into an aligned temporary array allocated on the stack and manually find the minimum of that array and store it on our result matrix.

Lastly, we changed the matrix elements from type *int* to *short*. The benefit of which is twofold: we can load twice the number of elements into the *SIMD* registers, and halving the size of *MPI* messages, making it faster.

3 Experiments

In concurrent[7] and parallel computing, theory only get us so far. The behaviour and efficiency of concurrent and parallel programs can be quite unpredictable—not just because the order of execution of actions between different processes is not guaranteed to go as we would like, but also thanks to peculiarities of the hardware, operating system, compiler, and so on. Therefore, thinking that an increase to the number of cores will directly lead efficiency all the way through the roof is nothing short of naïve.

[7] Also thought as potential parallelism. Maybe parallel computing is a sufficient designation.

Of the inputs that were provided, we benchmarked all of them with the exception of *Input5*, as it only works for 1 process and numbers of processes divisible by 5[8], given that the input size is 5 and 5 is a prime number. The only use that *Input5* would have is that it is a small input, but we have already got *Input6* so that aspect is covered. It could be useful to have a small input running for a large amount of processes, but the number of pages is of the essence. For each input, we show the measurement for the sequential implementation as well as for each of the requested number of processes: 1, 4, 9, 16, 25—the first 5 positive, non-zero, square numbers. The number of processes is represented by p, and the result is represented by $T(p)$, in milliseconds. We also did some calculations: $S(p)$, the speedup, and $S(p)/p$, the speedup divided by the number of processes used. We collected the data for our experimental analysis by running each input, for every number of processes for which the size is valid, 10 times each, and we make the average of the elapsed time of each combination of these two parameters (input and number of processes). Only the time spent running Fox's algorithm was measured.

The experimental analyses were made on a personal computer of ours, a desktop[9], and then on the distributed machines at *DCC* that were provided by the professor. In order to run more processes than our personal computer had to offer, we used *mpirun*'s *-oversubscribe* flag. It is possible that decreases in the runtime for a larger number of processes are linked to the number of processes surpassing the number of cores available.

3.1 Personal Computer

The results for our personal computer can be visualized in Table 3.1. The best absolute metrics, $T(p)$, were achieved using 4 processes, for smaller inputs such as 6 and 300, and using 9 processes, for larger inputs such as 600, 900 and 1200. At first, there is an improvement up until peak performance at 4 or 9 processes, but then $T(p)$ worsens and never recovers from such a tendency. The speedup follows the same exact pattern. However, the speedup per process always decreases whenever the number of processes, p, is increased for a given input. It should be noted that *Input1200* using 25 processes made a sudden improvement in performance when compared to its previous result, using 16 processes.

Given that the speedup per process is always at its best when running sequentially, we can conclude that either our implementation does not make an efficient usage of processes, or the input sizes are not large enough. On the bright side, some parallelism can help the code run up to three or four times faster than sequentially. In fact, by using 4 processes it is possible to reach nearly 1 of speedup per process.

3.2 Distributed Machines

The results for the distributed machines at *DCC* can be visualized in Table 3.2. To start off, our implementation runs, on average, between 5 and 20 times faster if done sequentially, when compared to any attempt at parallelism. Clearly, the overhead of the communication between the distributed machines outweighs the performance gain that we noticed using our personal computers. In spite of this, starting at $p = 4$, there is a slight improvement as the number of processes increases. Contrary to our personal computer, *Input1200* at 25 processes has an increase in the time spent running. Such is the way of outliers. The same can be said about *Input900* at 9 processes. In opposition to all other inputs, the runtime of *Input6* always worsens as the number of processes increases.

As we can see in Table 3.2, per-slot usually runs faster than per-node, which is to be expected, as per-slot distribution works by exhausting the slots of each machine before working with another one. On the other hand, per-node tries to distribute the processes

[8]Therefore, we could only run it using 1 and 25 processes.

[9]The specifics for the hardware are as follows: CPU: AMD Ryzen 5 7600X (12) @ 5.46 GHz; 16GB of RAM; On Arch Linux x86_64.

Sequential (ms)	Proc. (p)	1	4	9	16	25
Input6						
0.002	T(p) (ms)	0.0098	0.0218	0.0777	-	-
-	S(p)	1	0.44954	0.12613	-	-
-	S(p) / p	1	0.11239	0.01401	-	-
Input300						
9.4104	T(p) (ms)	9.6628	2.6653	2.8318	6.2449	9.0632
-	S(p)	1	3.62541	3.41225	1.54731	1.06616
-	S(p) / p	1	0.90635	0.37914	0.09671	0.04265
Input600						
43.9175	T(p) (ms)	45.7351	21.9325	14.9929	24.1841	36.7309
-	S(p)	1	2.08527	3.05045	1.89112	1.24514
-	S(p) / p	1	0.52132	0.33894	0.1182	0.04981
Input900						
159.355	T(p) (ms)	150.1436	43.8993	43.4473	60.8248	69.5229
-	S(p)	1	3.42018	3.45576	2.46846	2.15963
-	S(p) / p	1	0.85505	0.38397	0.15428	0.08639
Input1200						
319.1057	T(p) (ms)	308.3153	110.5511	77.5158	137.5359	110.7592
-	S(p)	1	2.78889	3.97745	2.24171	2.78365
-	S(p) / p	1	0.69722	0.44194	0.14011	0.11135

Table 3.1: Time run, speedup and speedup per process for inputs 6, 300, 600, 900 and 1200, using sequential code and 1, 4, 9, 16 and 25 processes for each. This experiment ran on one of our personal computers, a desktop[11].

equally amongst the machines specified on the hostfile. More machines incur in more communication through the network, which is slower than *IPC* (Inter-Process Communication), yet we can see some entries, such as the ones for *Input6* at 9 processes, where per-node is significantly faster than per-slot. Such outliers may be explained by network traffic, given every machine has a maximum of 8 slots as per the hostfile given to us. Thus, with 9 processes, atleast 2 machines are communicating. Another explanation that we came up with was the fact that the machines that were used are part of a cluster of machines made available by our faculty. As such, we don't have exclusivity of the resources of the machine, so another person might have been using it, resulting in a slower computation, and, generally, less stable measurements.

4 Conclusion

We faced some difficulties during the initial implementation of both Distance Product and Fox's algorithm. Fox's algorithm can be conceptually difficult to follow, although it is quite simple. Moreover, we have seen that there is a clear distinction between our personal computer, the per-node configuration and the per-slot configuration. Our personal computer was considerably more reliable than the distributed machines, as latency between processes is not such a big factor. Our heavily optimized implementation could have proven more useful if we had initially planned to compare it to non-optimized code, as it could have shown the impact sequential optimizations can have on parallel implementations.

Per-node						
Sequent. (ms)	Proc. (p)	1	4	9	16	25
Input6						
0.003	T(p) (ms)	0.0111	1.3575	1.7963	-	-
	S(p)	1	0.00818	0.00618	-	-
	S(p) / p	1	0.00205	0.00069	-	-
Input300						
11.1956	T(p) (ms)	12.0128	211.3598	141.1862	133.5182	126.0115
	S(p)	1	0.05684	0.08508	0.08997	0.09533
	S(p) / p	1	0.01421	0.00945	0.00562	0.00381
Input600						
59.6815	T(p) in ms	61.6077	988.8609	814.1723	591.4484	578.2680
	S(p)	1	0.0623	0.07567	0.10416	0.10654
	S(p) / p	1	0.01558	0.00841	0.00651	0.00426
Input900						
193.4166	T(p) (ms)	203.5334	2647.0701	3048.6654	1649.4749	1638.8013
	S(p)	1	0.07689	0.06676	0.12339	0.1242
	S(p) / p	1	0.01922	0.00742	0.00771	0.00497
Input1200						
414.9326	T(p) (ms)	440.6541	4499.8452	3705.4123	3670.2944	4110.3343
	S(p)	1	0.09793	0.11892	0.12006	0.10721
	S(p) / p	1	0.02448	0.01321	0.0075	0.00429
Per-slot						
Sequent. (ms)	Proc. (p)	1	4	9	16	25
Input6						
0.003	T(p) (ms)	0.0119	0.0579	50.7913	-	-
	S(p)	1	0.20553	0.00023	-	-
	S(p) / p	1	0.05138	0.00003	-	-
Input300						
11.1956	T(p) (ms)	12.0137	7.5779	111.0159	194.3714	216.5700
	S(p)	1	1.58536	0.10822	0.06181	0.05547
	S(p) / p	1	0.39634	0.01202	0.00386	0.00222
Input600						
59.6815	T(p) (ms)	61.6039	31.7235	546.8797	726.9047	970.9797
	S(p)	1	1.9419	0.11265	0.08475	0.06345
	S(p) / p	1	0.48548	0.01252	0.0053	0.00254
Input900						
193.4166	T(p) (ms)	203.8434	68.0081	1209.1462	2191.1903	2603.9552
	S(p)	1	2.99734	0.16858	0.09303	0.07828
	S(p) / p	1	0.74934	0.01873	0.00581	0.00313
Input1200						
414.9326	T(p) (ms)	438.0787	181.8829	3837.5861	4173.4084	4808.4144
	S(p)	1	2.40858	0.11415	0.10497	0.09111
	S(p) / p	1	0.60215	0.01268	0.00656	0.00364

Table 3.2: Time run, speedup and speedup per process for inputs 6, 300, 600, 900 and 1200, using sequential code and 1, 4, 9, 16 and 25 processes for each. This experiment was run on the provided distributed machines of *DCC*. The first sub-table was the result of executing using the per-node configuration, whereas the second sub-table was the result of executing using the per-slot configuration.